UPC

Departament d'Arquitectura de Computadors

Tema 3

Jerarquía de Memoria

Memoria Virtual

Prof. Montse Farreras
Curso 2023-2024

montserrat.farreras@upc.edu

Universitat Politècnica de Catalunya

Escola Politècnica Superior d'Enginyeria de Vilanova i la Geltrú
EPSEVG

UNIVERSITAT POLITÈCNICA
DE CATALUNYA
BARCELONATECH

3. Jerarquía de Memoria-Memoria Virtual

- 3.1. Introducción
- 3.2. Traducción de direcciones y paginación
- 3.3. Integrando memoria virtual y cache

3.1. Introducción

3.1.1. Revisión de la jerarquía de memoria

3.1.2. Introducción a la memoria virtual

3.1.1. Revisión de la jerarquía de memoria

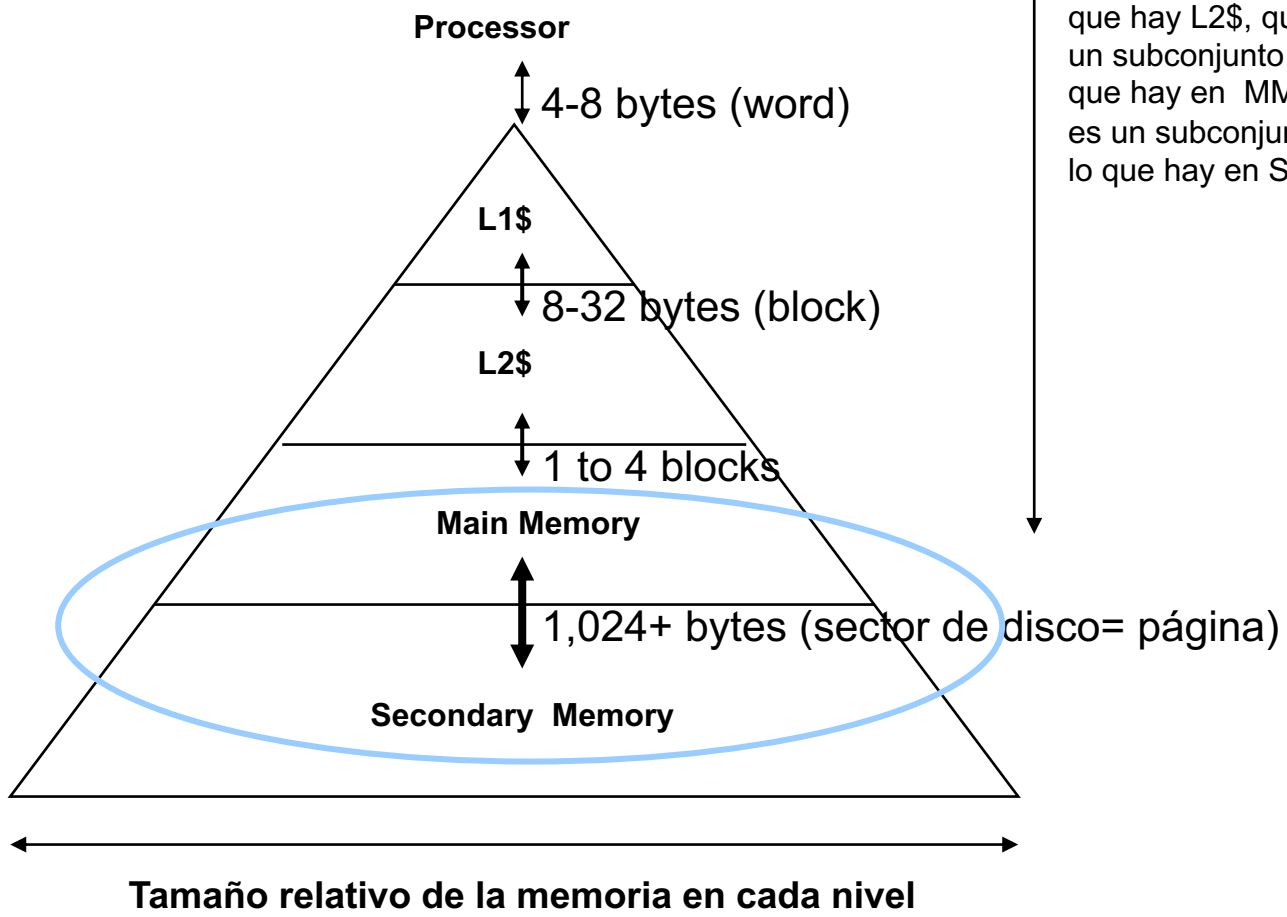
- Registros ↔ Memoria
  - Por el compilador (programador?)
- Cache ↔ Memoria Principal
  - Por el controlador de cache
- Memoria Principal ↔ Discos
  - Por el sistema operativo (**memoria virtual**)
  - La traducción de direcciones virtuales a direcciones físicas se realiza con la asistencia del hardware (MMU, Page Table, TLB, etc)
  - Por el programador (archivos)

3.1.1. Revisión de la jerarquía de memoria

Revisión de la jerarquía de memoria

Inclusivo
Lo que hay en L1$ es un subconjunto de lo que hay L2$, que es un subconjunto de lo que hay en MM, que es un subconjunto de lo que hay en SM

Incrementando distancia desde el procesador en tiempo de acceso



3.1.2. Introducción a la memoria virtual

- Usar la memoria principal como una “cache” para almacenamiento secundario (disco)
  - Manejado conjuntamente por el hardware de la **CPU** y el **sistema operativo** (OS).
  - Permite la **compartición eficiente y segura** de memoria **entre multiples programas.**
  - Proporciona la ejecución de programas más grandes que el tamaño de la memoria física.
  - Simplifica cargar un programa para ejecución porque **permite recolocación del código** (el código puede ser cargado en cualquier parte de la memoria principal).
- Qué hace que funcione? – otra vez el principio de localidad.
  - Un programa es probable que acceda a una parte relativamente pequeña de su espacio de direcciones durante cualquier período de tiempo.
- Los **programas comparten la memoria** principal.
  - **Cada uno tiene un espacio lógico de direcciones** con su código frecuentemente usado y datos.
  - Protegido de otros programas.

3.1.2. Introducción a la memoria virtual

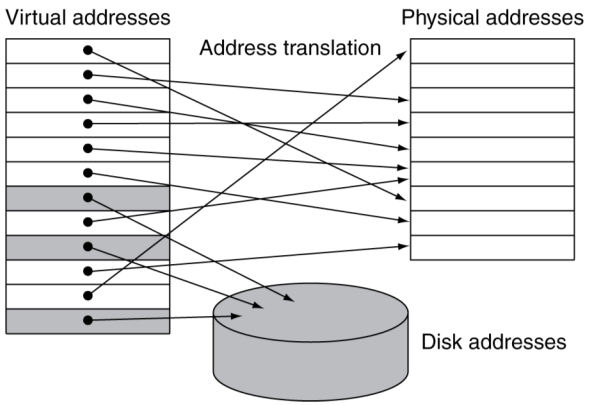
- Cada programa es compilado en su 'propio' espacio de direcciones – un espacio "lógico" de direcciones
  - Cada dirección lógica debe ser traducida a una dirección fisica (una dirección en la memoria principal) → A esta direccion lógica también la llamaremos virtual cuando la memoria física es menor al espacio de direcciones lógico (casi siempre).

  → se necesita una traducción → opción 3
  - OPCIÓN 1: **Asignación** de @ a instrucciones y datos **en tiempo de compilación**
    - @ física == @ lógica → No hay traducción en tiempo de ejecución
  - OPCIÓN 2: **Asignación** de @ a instrucciones y datos **en tiempo de carga**
    - Reubicación estática: **se reescribe el espacio de @ lógico en tiempo de carga**
    - @ físicas == @ lógicas → No hay traducción en tiempo de ejecución
    - Facilita carga en sistemas multiprogramados pero es poco flexible
  - OPCIÓN 3: **Asignación** de @ a instrucciones y datos **en tiempo de ejecución**
    - @ físicas != @ lógicas → requiere traducción en tiempo de ejecución
    - Procesos pueden cambiar de posición en memoria sin modificar su espacio lógico de @
    - Esquema de gestión de memoria simple con traducción: reubicación dinámica
- CPU y OS traducen las direcciones lógicas/virtuales a direcciones físicas
  - Lo que entre cache y MM era "**bloque**" aquí entre MM y VM se llama **página**
  - Lo que en cache era un "**miss**", en VM se llama **fallo de página** (page fault)

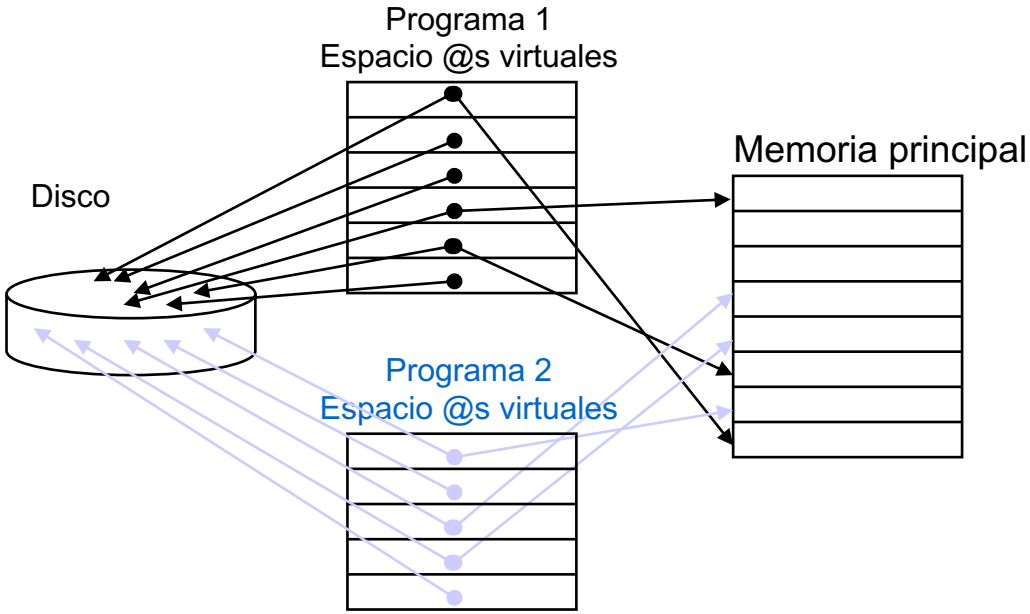
3.1.2. Introducción a la memoria virtual

- El espacio virtual de direcciones de un programa es dividido en páginas (todas de tamaño fijo) o segmentos (de tamaño variable)
  - La localización inicial de cada página (en memoria principal y en disco) está contenida en la tabla de páginas de programa (program's page table)

Espacio de direcciones de 1 programa



Espacio de direcciones de 2 programas

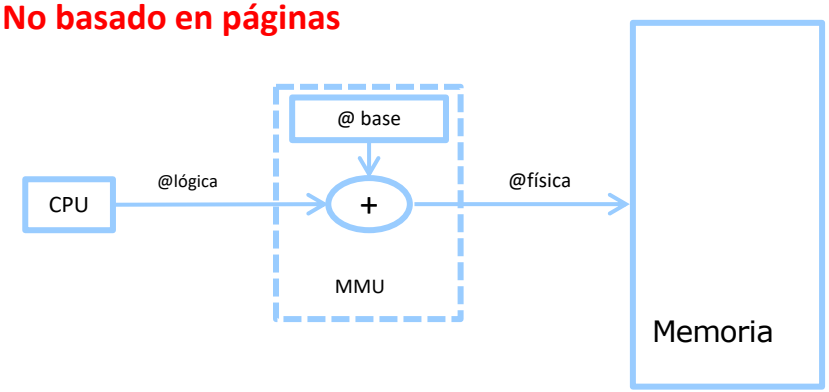


3.1.2. Introducción a la memoria virtual

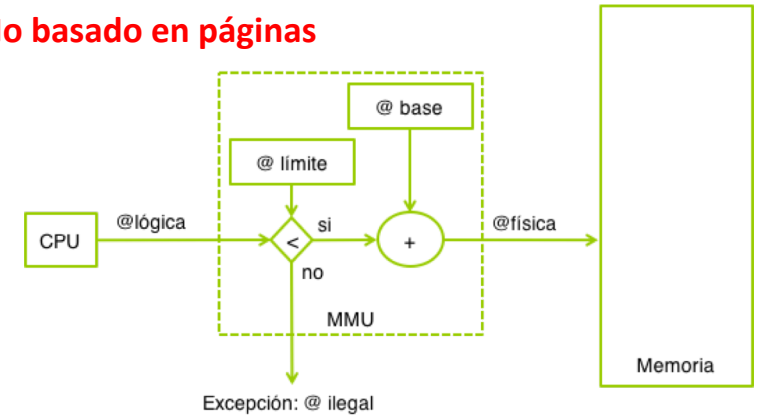
Opción 3: Asignación de @ a instrucciones y datos en tiempo de ejecución

De los métodos de traducción dinámicos (explicados en SO), explicaremos el soporte hardware para los métodos basados en páginas y segmentos.

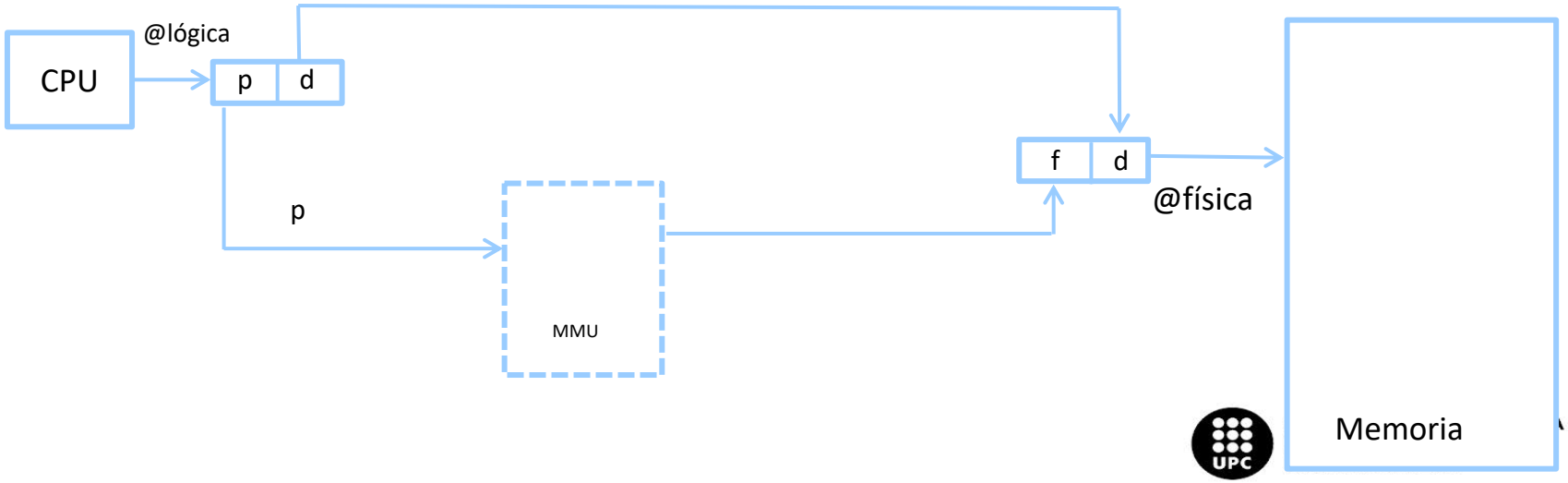
No basado en páginas



No basado en páginas



Basado en páginas



3.2. Paginación y traducción de direcciones

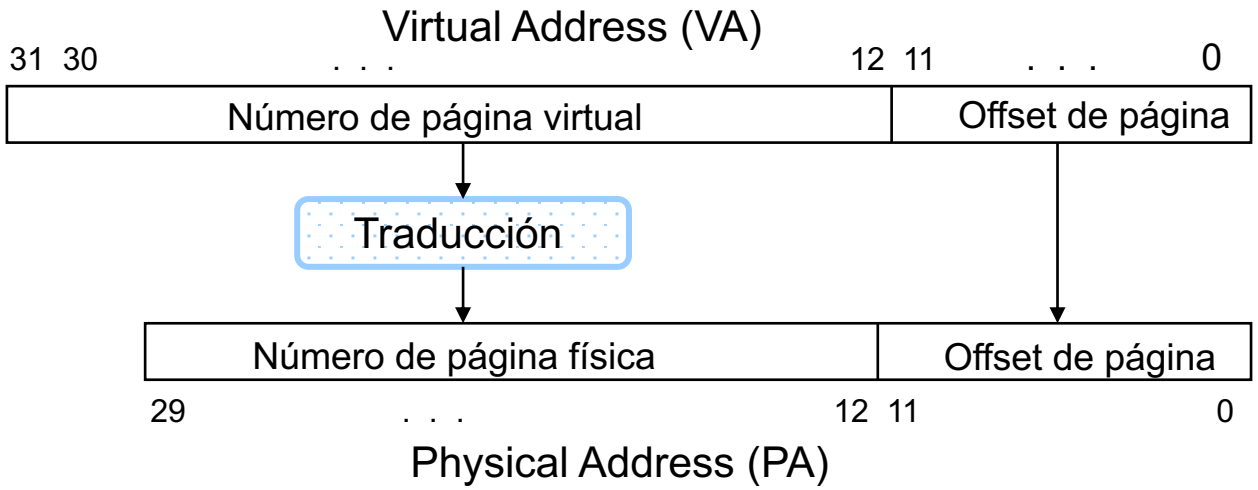
3.2.1. Direcciones virtuales-Direcciones físicas

3.2.2. Paginación

3.2.3. Translation Lookaside Buffers (TLB)

3.2.1 Direcciones virtuales- Direcciones físicas

- Una dirección virtual se traduce a una dirección física mediante una combinación de hardware y software → en este ejemplo cada página tiene 4K bytes = 2^{12} bytes

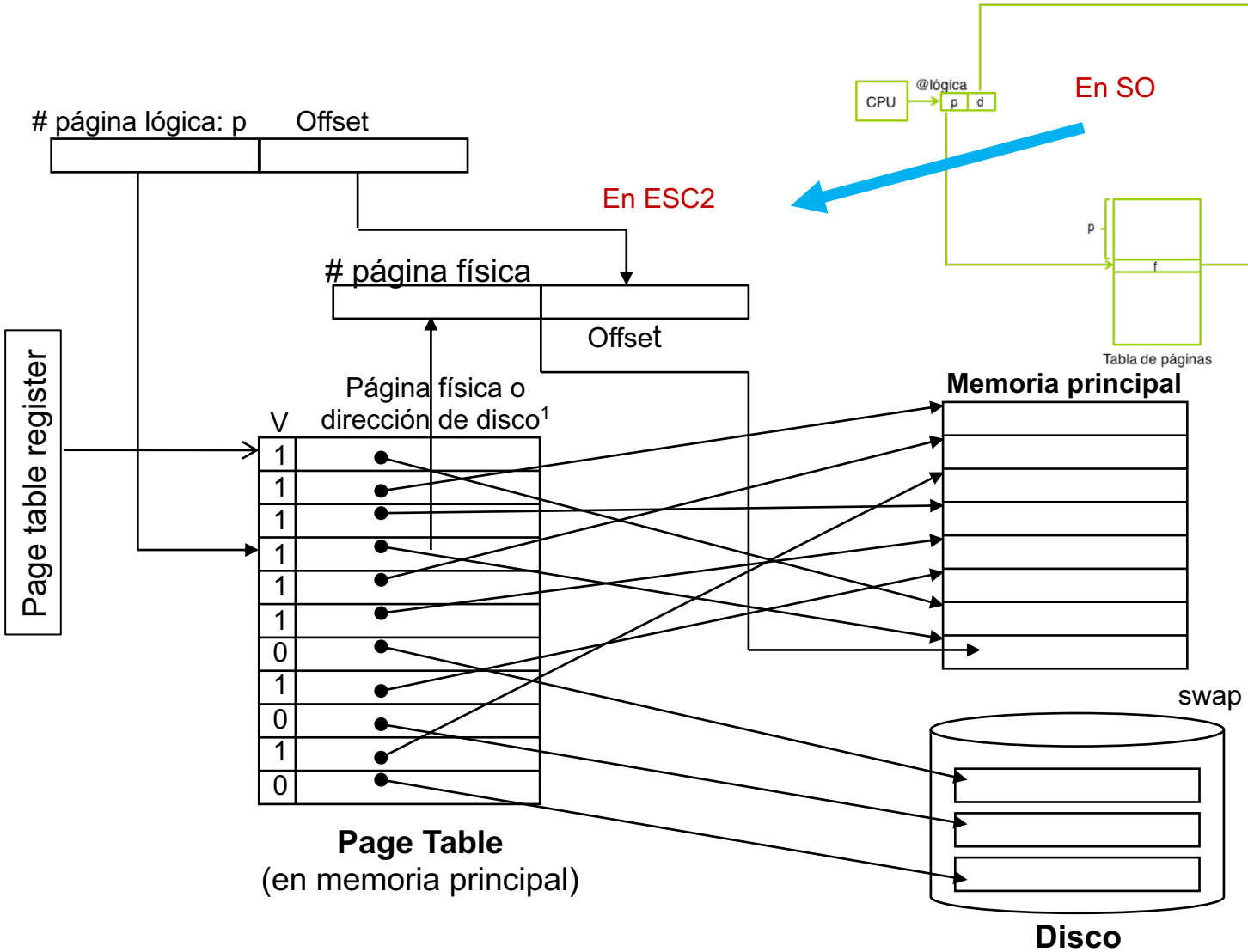


- Por lo tanto: cada petición de memoria requiere *primero* una traducción desde el espacio virtual al espacio físico.
  - Un fallo de memoria virtual (cuando la página no está en la memoria física), es un fallo de página

3.2.2. Paginación

- Las páginas deben ser grandes para amortizar el alto coste en tiempo de acceso: de 4 KBytes a 16 KBytes
- En caso de fallo de página, la página se tiene que traer desde disco
  - Tarda millones de ciclos de reloj
  - Es manejado por el sistema operativo
- En escritura nunca se utiliza Write Through en MV, pues tarda demasiado
- Tratar de minimizar el fallo de página
  - Emplazamientos completamente asociativos → tiene una dificultad: buscar la página, pues puede estar en cualquier lugar de la memoria → se utiliza la **Tabla de páginas** que nos dice si está y donde está (su traducción)
  - Algoritmos de reemplazo inteligentes

3.2.2 Paginación



Se aconseja la siguiente cantidad de memoria *swap*:

- 1 GB de RAM o menos: la misma cantidad para la *swap*
- entre 2 GB y 4 GB de RAM: la mitad para la *swap*
- más de 4 GB de RAM: 2 GB de *swap*

[1]Podríamos tener 2 tablas: una traduciendo a memoria principal y otra auxiliar a disco. En algunos sistemas está en la misma tabla

3.2.2 Paginación

Espacio swap

El espacio *swap* o de intercambio será normalmente una partición del disco, pero también puede ser un archivo. Los usuarios pueden crear un espacio de intercambio durante la instalación o en cualquier momento posterior, en caso de ser necesario.
El espacio de intercambio es generalmente recomendado a los usuarios con menos de 1 GB de RAM.

Una partición *swap* se puede crear con la mayoría de las herramientas que gestionan las particiones en GNU/Linux (por ejemplo, fdisk, cfdisk).

Cuando Windows usa toda la RAM disponible, empieza a utilizar la memoria virtual para tener más recursos de forma temporal. Windows administra de manera automática el tamaño de ese archivo (archivo de paginación).

En Windows 10, para modificar el tamaño de la memoria virtual se utiliza configuración avanzada del sistema en el panel de control.

3.2.2 Paginación

Tabla de páginas (Page Table)

- Una entrada para cada página
- Una tabla por proceso
- Suele guardarse en memoria
- Contiene información de emplazamiento
  - La Page Table es indexada con el número de página virtual. Cada posición en la tabla es una *Entrada en la Tabla de Páginas (**PTE**)*
  - Page table register (registro de página de tabla) en la CPU apunta a la primera dirección de la tabla de páginas
- Si la página está presente en la memoria física
  - PTE almacena el número de página física (marco o frame)
  - Así como otros bits de estado (validez, referenced, dirty, …)
- Si una página no está presente en la Tabla de Páginas
  - PTE puede referirse a una localización en la zona de swap de disco
    - En el ejemplo de la figura anterior marcados con el bit V a 0 son páginas que no están en memoria, y apuntan a la zona de disco SWAP
    - Pero podría implementarse de otra manera

3.2.2 Paginación

Fallo de página (Page Fault)

- Si el bit de validez al acceder a la tabla de páginas está a 0, la página no se encuentra en la memoria principal.
- Esto transfiere el control al sistema operativo, quien tiene que encontrar la página en el siguiente nivel de la jerarquía (disco) y copiarla en memoria principal.
- Si todos los frames de página en la memoria principal están 'ocupados' (en uso), y no hay espacio para introducir una nueva, el SO ha de elegir una a reemplazar:
  - Usualmente se utiliza la política de LRU
  - Las páginas reemplazadas se escriben en la zona de swap de disco
- Implementar LRU completamente es caro, en términos de hardware (con estructura de datos (lista) para almacenar las páginas en orden de más a menos recientemente utilizada).
- Algunos computadores simplifican el LRU:
  - 1 sólo bit por entrada, ***referenced bit*** (o *use bit*), cada vez que se accede a la página se pone a 1. El SO periódicamente lo vuelve a poner a 0
    - → El SO puede saber así qué páginas no han sido referenciadas en un determinado periodo.
    - → La página seleccionada para ser reemplazada será una de las que tenga el *referenced bit* a 0

3.2.2 Paginación y traducción de direcciones

Escrituras

- Qué pasa con las escrituras?
  - En escritura casi nunca se utiliza ***Write Through*** en Memoria Virtual. Tarda demasiado
  - Por lo tanto, debe implementar una política **write-back** → escribiendo sólo en la página en memoria principal y actualizando ésta información en disco, cuando la página es reemplazada.

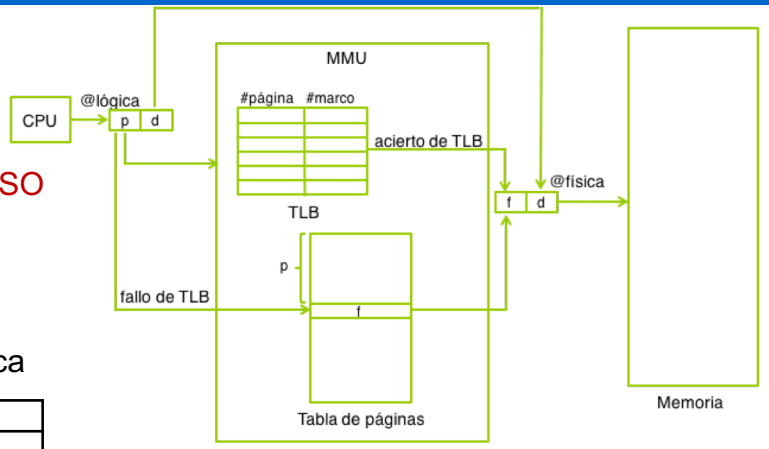
3.2.3 Translation Lookaside Buffers (TLB)

Queremos hacer más rápida la traducción de direcciones porque:

- ✓ La traducción de direcciones requeriría 2 accesos a memoria:
  - ✓ Uno para acceder a la tabla de páginas
  - ✓ El otro para el acceso a memoria propiamente
- ✓ El acceso a la tabla de páginas tiene una buena localidad
  - ➢ Por lo tanto, usar una **cache rápida** de las entradas de traducciones recientemente utilizadas
  - ➢ **Translation Look-aside Buffer (TLB):** Memoria asociativa de acceso más rápido en la que se almacena la traducción para las páginas *activas*
    - ✓ Tiene una parte de las traducciones de página → no todas
  - ➢ Típico: 16–512 PTEs (entradas), 0.5–1 ciclos en acierto, 10–100 ciclos por fallo, 0.01%–1% tasa de fallos
  - ➢ Los fallos pueden ser manejados por el software, el hardware o ambos

3.2.3 Translation Lookaside Buffers (TLB)

Tabla de páginas con TLB



En SO

En ESC2

TLB

página lógica: p

V	D	R[1]	Tag	Página física
1				
1				
1				
0				
1				

Page table register

V	D	R[1]	Página física o dirección de disco
1			
1			
1			
1			
1			
1			
0			
1			
0			
1			
0			

Page Table
en memoria principal

Memoria principal

Disco

[1]V: validez, D: dirty, R:referenced (usado como LRU simple)

3.2.3 Translation Lookaside Buffers (TLB)

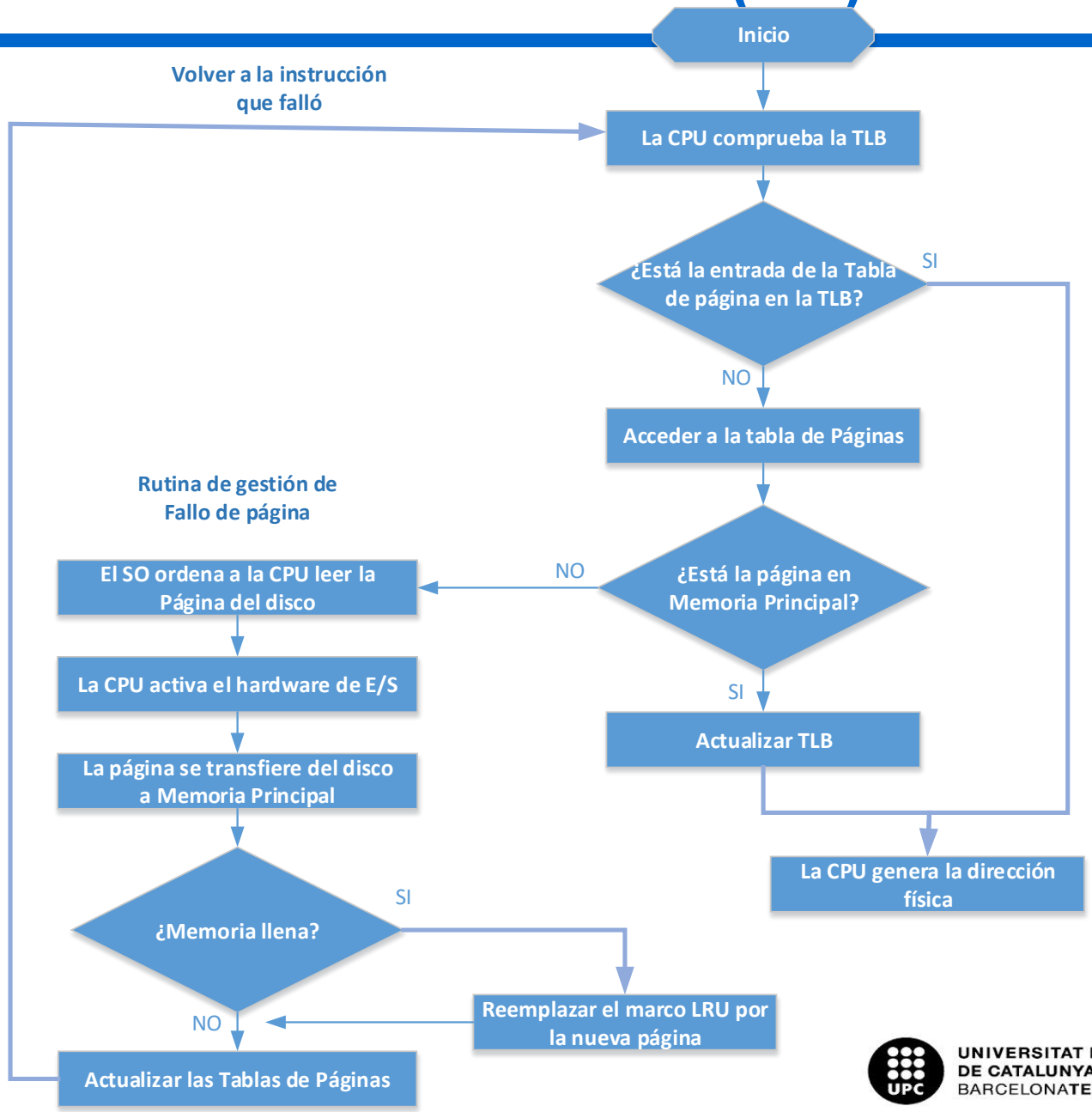
TLB hits:

- El número de página almacenado en el TLB es usado para formar la dirección
- El bit de **ref** se pone a 1, y el de **dirty** si es una escritura a 1 también

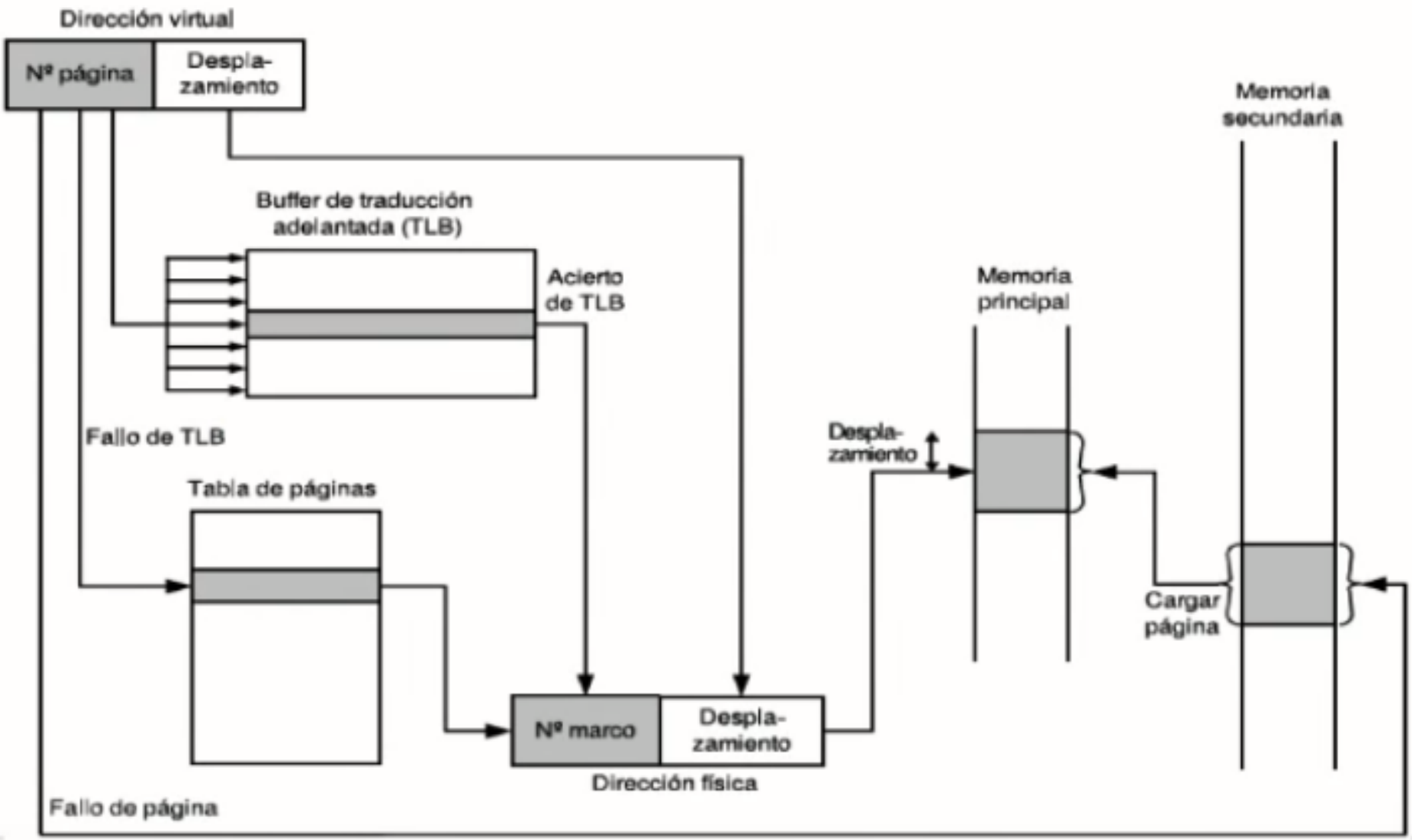
TLB Misses:

- Si la página está en memoria principal→ tan solo es un TLB miss
  - ✓ Cargar la traducción de página desde la Page Table al TLB, y sus correspondientes bits
  - ✓ Una entrada del TLB se ha de borrar para poner la nueva entrada. En algunas ocasiones si se ha usado write-back (para bits dirty y ref), estos se han de actualizar en la page table.
  - ✓ Puede ser manejado en software o hardware
  - ✓ Si la página no está en memoria → page fault verdadero
    - ✓ OS se ocupa de coger la página de disco, cargarla en memoria y actualizar la tabla de páginas.
    - ✓ Previamente puede tener que actualizar la página que 'echa' fuera de memoria en disco (si tiene el bit dirty=1).

3.2.3 Translation Lookaside Buffers (TLB)



3.2.3 Translation Lookaside Buffers (TLB)



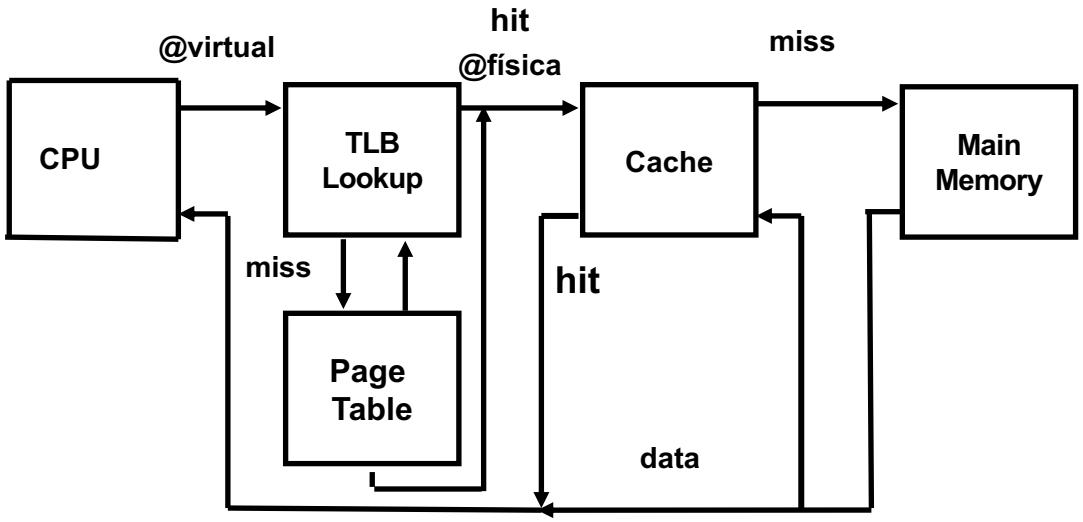
3.3 Integrando memoria virtual y cache

3.3.1. TLB y cache

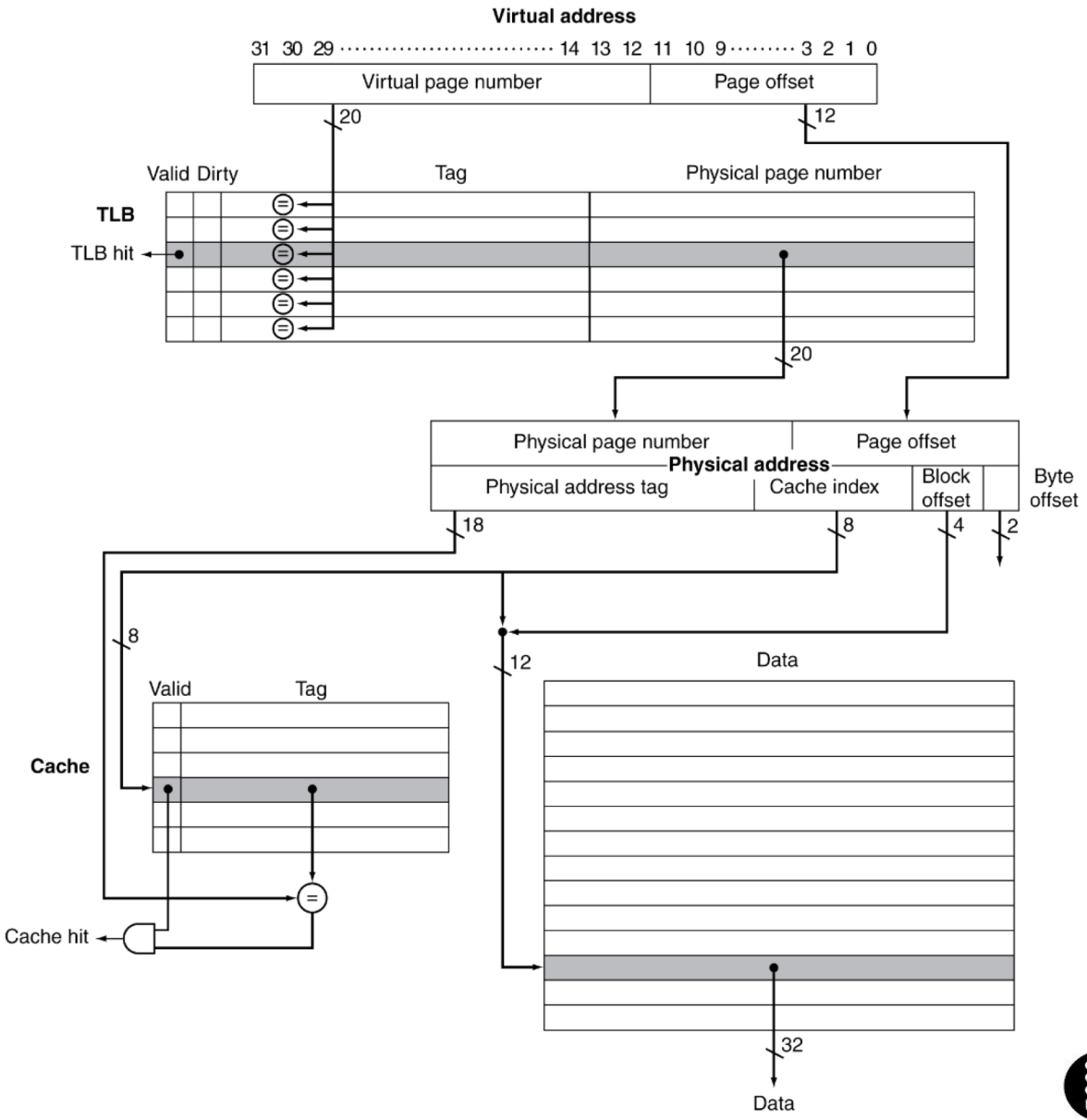
3.3.2. Tabla de páginas en varios niveles

3.3 Integrando memoria virtual y cache

La cache también funciona con direcciones físicas. Por tanto, primero hay que traducir las @ virtuales a @ físicas de cache, por medio de la **TLB**.



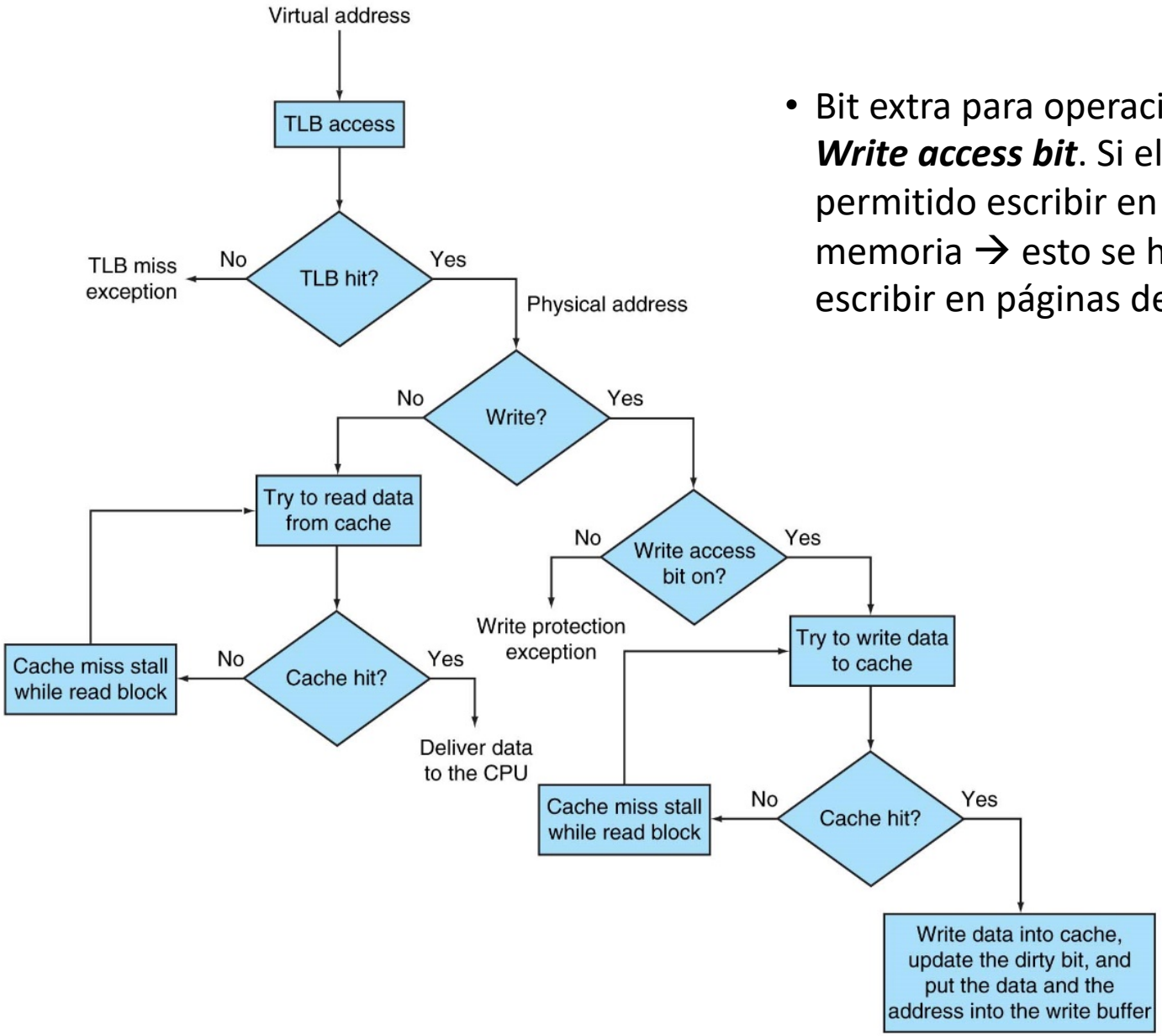
3.3 Integrando memoria virtual y cache



3.3.1 TLB y cache

TLB	Page Table	Cache	Es posible? En qué circunstancias?
Hit	Hit	Hit	Sí, es lo que queremos!
Hit	Hit	Miss	Sí, aunque no comprobaríamos la Page Table si el TLB acierta
Miss	Hit	Hit	Sí – TLB miss (se genera excepción). Traducción en Page Table
Miss	Hit	Miss	Sí – TLB miss (se genera excepción). Traducción en Page Table, pero el dato no está en cache.
Miss	Miss	Miss	Sí – Fallo de página (genera excepción).
Hit	Miss	Miss/ Hit	Imposible – la traducción en TLB no es posible si la página no está en memoria
Miss	Miss	Hit	Imposible – dato no colocado en cache si la página no está en memoria

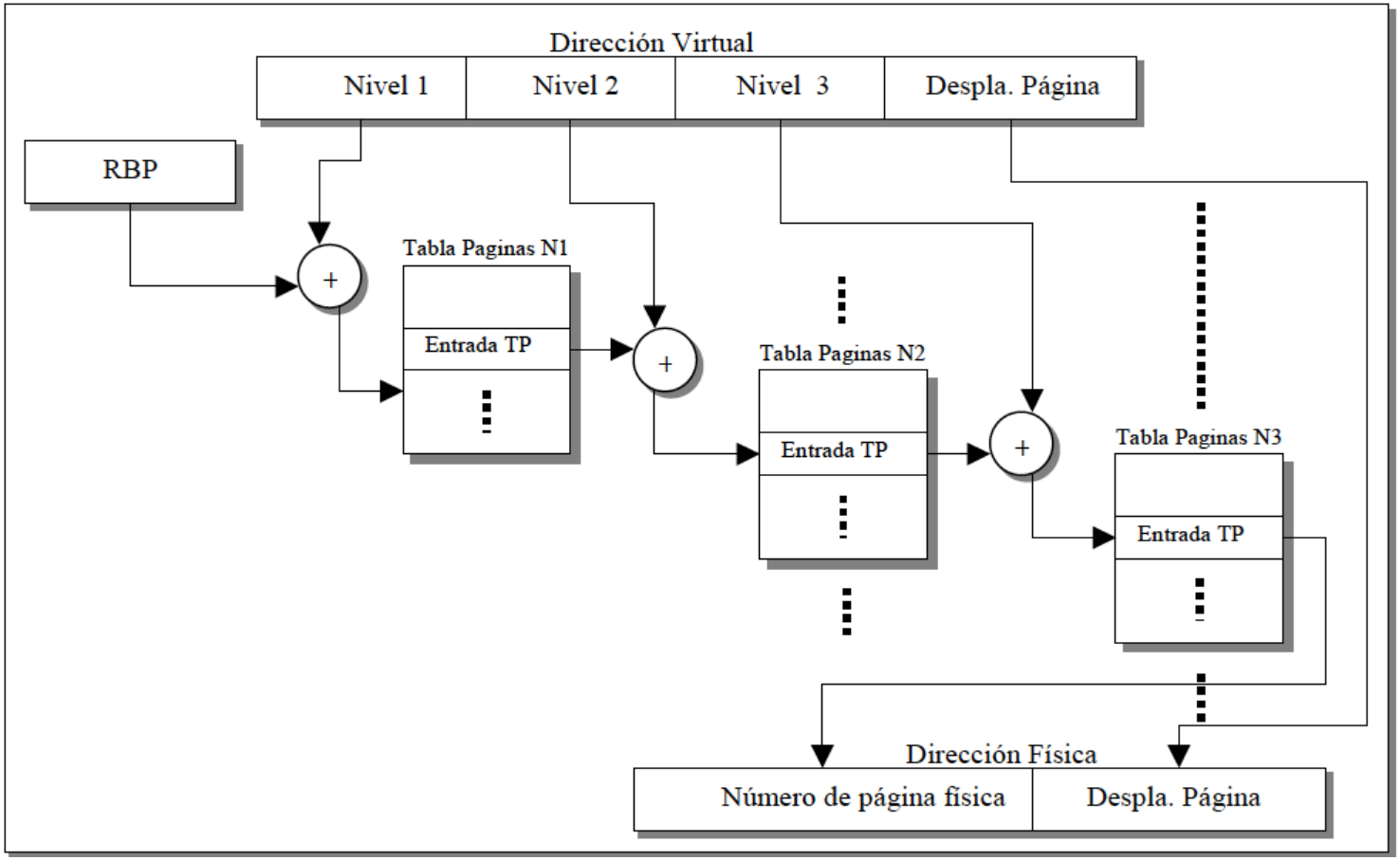
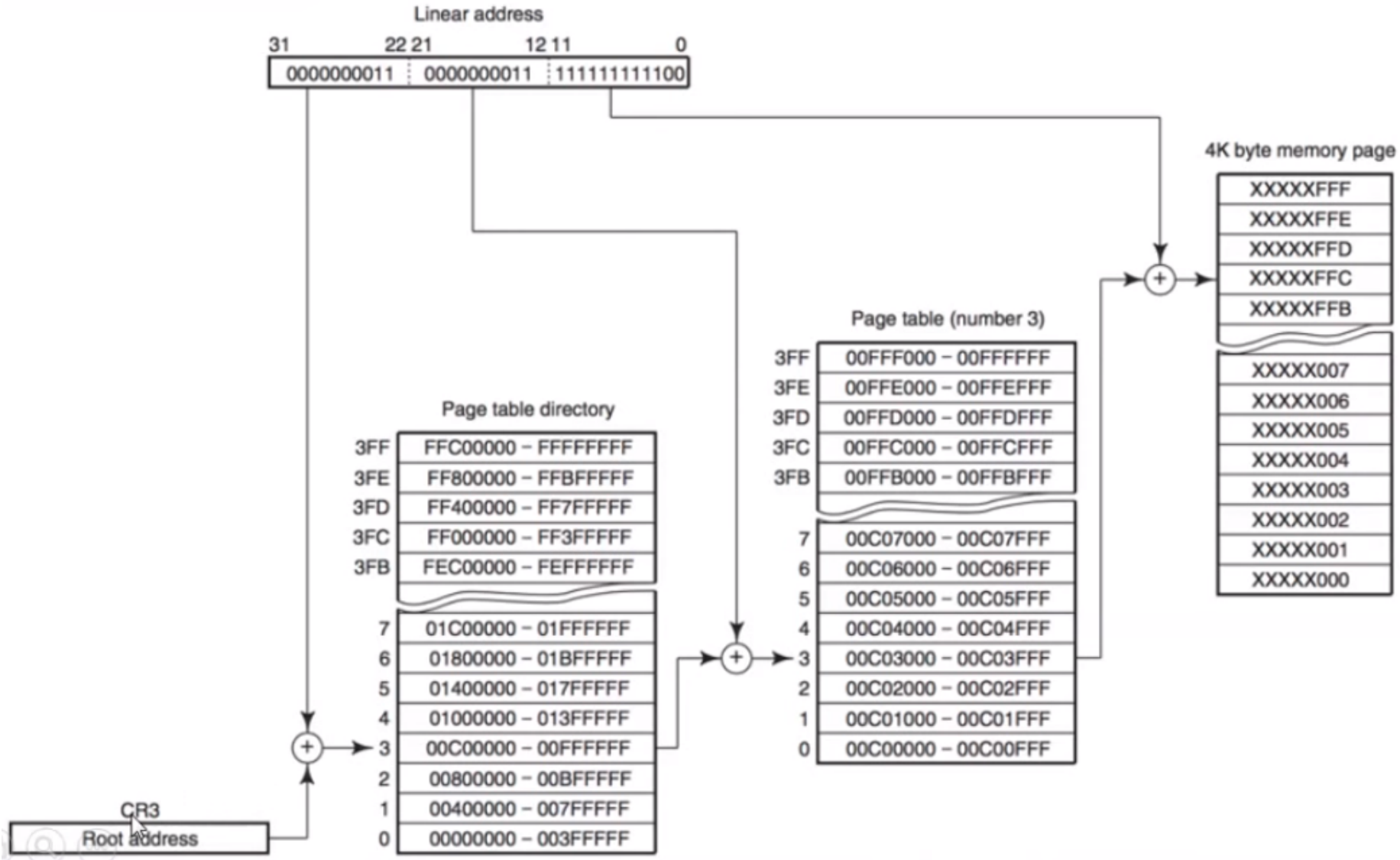
3.3.1 TLB y cache



- Bit extra para operaciones de escritura: ***Write access bit***. Si el bit está a 0 no está permitido escribir en esa posición de memoria → esto se hace para prevenir escribir en páginas de sólo lectura

3.3.2 Tabla de Páginas en varios niveles

La longitud típica de la tabla de páginas se limita al tamaño de página



3.3.2 Tabla de Páginas en varios niveles